Aufgabe 1

1.1)
a) Die Generalisierung ist {partiell, disjunkt}

b)

Skilift(LiftNr, Bezeichnung, Punkte, Fahrzeit)

Gondel(LiftNr, MaxLast, Panorama)

Sessellift(LiftNr, Sitze)

hat(Sessellift.LiftNr, STID)

Sitztyp(STID, Polsterung, Heizung)

c)
Es handelt sich um das Partitionenmodell.
Vorteile:
- keine Redundanzen, daher geringer Speicherbedarf. Wenn ein neuer Wert eingefügt wird, muss er nur an einer Stelle eingefügt werden.
- einfache Übersicht der Beziehungen durch ER-Diagramm
- einfach erweiterbar (z.B. durch die im Bild abgeschnittene Relation)

1.2)

a) Die Generalisierung ist {partiell, überlappend} (weil nicht alle Mitarbeiter dem Außendienst oder Manager angehören, aber es Manager*innen im Außendienst gibt)

b)
Mitarbeiter(MitarbeiterNr, Vorname, Nachname,
GebDatum, Gehalt, EinstellungsDatum,
IstAußendienst, IstManager,
Dienstwagen, VorgesetzterNr)

c) Hierbei handelt es sich um das Einrelationenmodell.
Vorteile:
- besonders performant können Abfragen zu einzelnen Mitarbeitenden gemacht werden
- bei diesem Beispiel ist die Vererbungshirarchie recht flach und es kommt nur ein einzelnes Attribut hinzu (beim Außendienst). Eine eigene Tabelle Manager zu haben, z.B. nur um Manager zu identifizieren ist fast mehr overhead, als einfach einen boolean dazu zu speichern.

Aufgabe 2

2.1

Code

```
SELECT store_id, COUNT(staff_id)
FROM store
LEFT JOIN staff USING (store_id) -- alle stores bleiben erhalten
GROUP BY store_id
HAVING COUNT(staff_id) <> 1;
```

Ausgabe

	STORE_ID	COUNT(STAFF_ID)
1	5	4
2	6	0

2 selected

2.2

Code

```
SELECT first_name, last_name, customer_id
FROM customer
      JOIN rental USING (customer_id)

MINUS

SELECT c.first_name, c.last_name, c.customer_id
FROM customer c
      JOIN rental rent ON c.customer_id = rent.customer_id
      JOIN inventory invent ON rent.inventory_id = invent.inventory_id
WHERE c.store_id = invent.store_id;
```

Ausgabe

	FIRST_NAME	LAST_NAME	CUSTOMER_ID
1	CODY	NOLEN	544
2	ERNEST	STEPP	384
3	FLOYD	GANDY	465
4	KRISTEN	CHAVEZ	194
5	LEWIS	LYMAN	478
6	PENNY	NEAL	271
7	RONNIE	RICKETTS	456

7 selected

2.3

Code

```
SELECT first_name, last_name
FROM customer
      JOIN rental USING (customer_id)
      JOIN inventory USING (inventory_id)
      JOIN film_category USING (film_id)
      JOIN category USING (category_id)
GROUP BY customer_id, first_name, last_name
HAVING COUNT(CASE WHEN name = 'Family' THEN 1 END) > 0
      AND COUNT(CASE WHEN name = 'Horror' THEN 1 END) > 2 * COUNT(CASE
WHEN name = 'Family' THEN 1 END);
```

Ausgabe

	FIRST_NAME	LAST_NAME
1	LEE	HAWKS
2	VERONICA	STONE
3	BARBARA	JONES
...		
27	LEO	EBERT

27 selected

2.4

Code

```
SELECT first_name, last_name
FROM actor
      JOIN film_actor USING (actor_id)
      JOIN film_category USING (film_id)
GROUP BY first_name, last_name, actor_id
HAVING COUNT(DISTINCT category_id) > 10;
```

Ausgabe

	FIRST_NAME	LAST_NAME
1	TOM	MCKELLEN
2	JOHNNY	CAGE
3	NICK	WAHLBERG
4	JOE	SWANK
...		
188	SPENCER	PECK

188 selected

Aufgabe 3

3.1

Code

```
SELECT verk_fil.store_id,
       verk.first_name,
       verk.last_name,
       COUNT(DISTINCT rent.rental_id), -- distinct, falls es mehrere Zahlungseinträge für die selbe Mietung gibt
       COALESCE(SUM(pay.amount), 0)
FROM staff verk
       JOIN rental rent ON verk.staff_id = rent.staff_id
       JOIN inventory inv ON rent.inventory_id = inv.inventory_id
       JOIN store verk_fil ON inv.store_id = verk_fil.store_id
       LEFT JOIN payment pay ON rent.rental_id = pay.rental_id
GROUP BY verk.first_name, verk.last_name,
       verk.staff_id, verk_fil.store_id;
```

Ausgabe

	STORE_ID	FIRST_NAME	LAST_NAME	COUNT(DISTINCTRENT.RENTAL_ID)	COALESCE(SUM(PAY.AMOUNT),0)
1	2	Jon	Stephens	1021	7320.81
2	6	Mike	Hillyer	1017	7544.96
3	6	Frank	Delfino	163	1154.47
4	6	Stephen	Miller	146	1095.14
...					
36	4	Frank	Delfino	154	1166.36

36 selected

3.2

Code

```
SELECT first_name, last_name, customer_id,
       COUNT(CASE WHEN name = 'Horror' THEN 1 END) AS ct_horror
FROM customer
       JOIN rental USING (customer_id)
       JOIN inventory USING (inventory_id)
       JOIN film_category USING (film_id)
       JOIN category USING (category_id)
GROUP BY customer_id, first_name, last_name
HAVING COUNT(CASE WHEN name = 'Horror' THEN 1 END) >= 4
ORDER BY COUNT(CASE WHEN name = 'Horror' THEN 1 END) DESC;
```

Ausgabe

	FIRST_NAME	LAST_NAME	CUSTOMER_ID	CT_HORROR
1	DUANE	TUBBS	513	5
2	KEN	PREWITT	563	5
3	EMMA	BOYD	134	5
4	ROSEMARY	SCHMIDT	204	5

...

34	EDGAR	RHOADS	507	4

34 selected

3.3

Code

```
SELECT first_name, last_name, COUNT(rental_id) AS total_number
FROM customer
JOIN rental USING (customer_id)
WHERE store_id = 6
GROUP BY GROUPING SETS((first_name, last_name, customer_id), ())
ORDER BY COUNT(rental_id);
```

Ausgabe

	FIRST_NAME	LAST_NAME	TOTAL_NUMBER
1	DELORES	HANSEN	21
2	ANDREA	HENDERSON	22
3	LORETTA	CARPENTER	22
4	MARIAN	MENDOZA	23

...

32	(null)	(null)	868

32 selected

Aufgabe 4

4.1 -- mit Rollup

Code

```
SELECT name AS category_name,
       rating,
       COUNT(film_id) AS film_count,
       SUM(length) AS total_length
FROM film
       JOIN film_category USING (film_id)
       JOIN category USING (category_id)
WHERE name IN ('Comedy', 'Music')
GROUP BY ROLLUP(rating, (category_id, name));
```

Ausgabe

	CATEGORY_NAME	RATING	FILM_COUNT	TOTAL_LENGTH
1	Comedy	G	11	1364
2	Music	G	2	132
3	(null)	G	13	1496
4	Comedy	R	8	950
5	Music	R	11	1088
6	(null)	R	19	2038
7	Comedy	PG	16	1869
8	Music	PG	10	1187
9	(null)	PG	26	3056
10	Comedy	NC-17	11	1255
11	Music	NC-17	20	2260
12	(null)	NC-17	31	3515
13	Comedy	PG-13	12	1280
14	Music	PG-13	8	1129
15	(null)	PG-13	20	2409
16	(null)	(null)	109	12514

16 selected

966.051 ms

Search

Operation	Object Name	Options	Starts	E-Rows	A-Rows	A-Time (ms)	Buffers	Cost
SELECT STATEMENT			1		16	0.891	44	16
SORT		GROUP BY R…	1	10	16	0.891	44	16
HASH JOIN			1	125	109	0.822	44	15
Access Predicates								
"FILM"."FILM_ID"="FILM_CATEGORY"."FILM_ID"								
NESTED LOOPS			1	125	109	0.408	14	15
NESTED LOOPS			1		109	0.4	14	
STATISTICS COLLECTOR			1		109	0.39	14	
HASH JOIN			1	125	109	0.367	14	6
Access Predicates								
"FILM_CATEGORY"."CATEGORY_ID"="CATE								
TABLE ACCESS	CATEGORY	FULL	1	2	2	0.043	6	3
Filter Predicates								
("CATEGORY"."NAME"='Comedy' OR "C								
INDEX	FILM_CATEGORY	FAST FULL …	1	1000	1000	0.096	8	3
INDEX	FILM_PK	UNIQUE SCAN						
Access Predicates								

Rollup verursacht hier Kosten von 16.

4.2 -- mit grouping sets

Code

```
SELECT name AS category_name,
    rating,
    COUNT(film_id) AS film_count,
    SUM(length) AS total_length
FROM film
    JOIN film_category USING (film_id)
    JOIN category USING (category_id)
WHERE name IN ('Comedy', 'Music')
GROUP BY GROUPING SETS((rating), (category_id, name, rating), ());
```

Ausgabe

	CATEGORY_NAME	RATING	FILM_COUNT	TOTAL_LENGTH
1	Comedy	G	11	1364
2	Music	G	2	132
3	(null)	G	13	1496
4	Comedy	R	8	950
5	Music	R	11	1088
6	(null)	R	19	2038
7	Comedy	PG	16	1869
8	Music	PG	10	1187
9	(null)	PG	26	3056
10	Comedy	NC-17	11	1255
11	Music	NC-17	20	2260
12	(null)	NC-17	31	3515
13	Comedy	PG-13	12	1280
14	Music	PG-13	8	1129
15	(null)	PG-13	20	2409
16	(null)	(null)	109	12514

16 selected

Operation	Object Name	Options	Starts	E-Rows	A-Rows	A-Time (ms)	Buffers	Cost
SELECT STATEMENT			1		16	0.899	44	16
SORT		GROUP BY R…	1	10	16	0.899	44	16
HASH JOIN			1	125	109	0.846	44	15
Access Predicates								
"FILM"."FILM_ID"="FILM_CATEGORY"."FILM_ID"								
NESTED LOOPS			1	125	109	0.403	14	15
NESTED LOOPS			1		109	0.392	14	
STATISTICS COLLECTOR			1		109	0.385	14	
HASH JOIN			1	125	109	0.362	14	6
Access Predicates								
"FILM_CATEGORY"."CATEGORY_ID"="CATE								
TABLE ACCESS	CATEGORY	FULL	1	2	2	0.047	6	3
Filter Predicates								
("CATEGORY"."NAME"='Comedy' OR "C								
INDEX	FILM_CATEGORY	FAST FULL …	1	1000	1000	0.079	8	3
INDEX	FILM_PK	UNIQUE SCAN						

Auch bei den Grouping sets werden Kosten von 16 verursacht. Auch der Ausführungsplan ist gleich, die Datenbank macht intern also (wahrscheinlich) genau das gleiche.

4.3 -- mit UNION ALL

Code

```
-- rating
SELECT NULL AS category_name,
       rating,
       COUNT(film_id) AS film_count,
       SUM(length) AS total_length
FROM film
       JOIN film_category USING (film_id)
       JOIN category USING (category_id)
WHERE name IN ('Comedy', 'Music')
GROUP BY rating

UNION ALL

-- category und rating
SELECT name AS category_name,
       rating,
       COUNT(film_id) AS film_count,
       SUM(length) AS total_length
FROM film
       JOIN film_category USING (film_id)
       JOIN category USING (category_id)
WHERE name IN ('Comedy', 'Music')
GROUP BY category_id, name, rating

UNION ALL

-- total_length
SELECT NULL AS category_name,
       NULL AS rating,
       COUNT(film_id) AS film_count,
       SUM(length) AS total_length
FROM film
       JOIN film_category USING (film_id)
       JOIN category USING (category_id)
WHERE name IN ('Comedy', 'Music');
```

Ausgabe

	CATEGORY_NAME	RATING	FILM_COUNT	TOTAL_LENGTH
1	(null)	PG-13	20	2409
2	(null)	R	19	2038
3	(null)	NC-17	31	3515
4	(null)	G	13	1496
5	(null)	PG	26	3056
6	Music	NC-17	20	2260
7	Music	PG-13	8	1129
8	Comedy	PG	16	1869
9	Music	R	11	1088
10	Music	PG	10	1187
11	Comedy	G	11	1364
12	Comedy	NC-17	11	1255
13	Music	G	2	132
14	Comedy	PG-13	12	1280
15	Comedy	R	8	950
16	(null)	(null)	109	12514

16 selected

886.744 ms

Search

Operation	Object Name	Options	Starts	E-Rows	A-Rows	A-Time (ms)	Buffers	Cost
SELECT STATEMENT			1		16	2.095	132	47
UNION-ALL			1		16	2.095	132	
HASH		GROUP BY	1	5	5	0.833	44	16
HASH JOIN			1	125	109	0.73	44	15
Access Predicates								
"FILM"."FILM_ID"="FILM_CATEGORY								
HASH JOIN			1	125	109	0.367	14	6
Access Predicates								
"FILM_CATEGORY"."CATEGORY_ID"								
TABLE ACCESS	CATEGORY	FULL	1	2	2	0.051	6	3
Filter Predicates								
("CATEGORY"."NAME"='Comedy'								
INDEX	FILM_CATEG	FAST FU…	1	1000	1000	0.089	8	3
TABLE ACCESS	FILM	FULL	1	1000	1000	0.138	30	9
HASH		GROUP BY	1	10	10	0.577	44	16
HASH JOIN			1	125	109	0.493	44	15

Hier sind die Kosten wesentlich höher, die Datenbank kann das nicht so gut optimieren. Für alle 3 groupbys wird hier ein eigenes SELECT statement verwendet, die Datenbank muss alle Tabellen einzeln zusammenjoinen, wodurch sich der Aufwand fast verdreifacht.

Aufgabe 5

5.1

Code

```
SELECT manager_staff_id,
       store_id,
       staff_id,
       SUM(amount) AS erloes
FROM store
       JOIN staff USING (store_id)
       JOIN payment USING (staff_id)
GROUP BY GROUPING SETS (
       (manager_staff_id, store_id, staff_id),
       (manager_staff_id, store_id),
       (store_id, staff_id)
);
```

Ausgabe

	MANAGER_STAFF_ID	STORE_ID	STAFF_ID	ERLOES
1	1	1	1	44694.83
2	2	2	2	44551.46
3	3	3	3	6620.07
4	4	4	4	6657.14
5	5	5	5	6662.75
6	5	5	6	6454.64
7	4	4	(null)	6657.14
8	2	2	(null)	44551.46
9	5	5	(null)	13117.39
10	1	1	(null)	44694.83
11	3	3	(null)	6620.07
12	(null)	1	1	44694.83
13	(null)	2	2	44551.46
14	(null)	3	3	6620.07
15	(null)	4	4	6657.14
16	(null)	5	5	6662.75
17	(null)	5	6	6454.64

17 selected

5.2

Code

```
SELECT country AS land,
       EXTRACT(YEAR FROM payment_date) AS jahr,
       name AS kategorie,
       COUNT(payment_id) AS anzahl,
       SUM(amount) AS summe
FROM payment
       JOIN rental USING (rental_id, customer_id)
       JOIN customer USING (customer_id)
       JOIN address USING (address_id)
       JOIN city USING (city_id)
       JOIN country USING (country_id)
       JOIN inventory USING (inventory_id)
       JOIN film_category USING (film_id)
       JOIN category USING (category_id)
WHERE name IN ('Family', 'Children', 'Travel')
GROUP BY GROUPING SETS (
       (country, EXTRACT(YEAR FROM payment_date)),
       (country, name),
       (country)
)
ORDER BY land;
```

Ausgabe

	LAND	JAHR	KATEGORIE	ANZAHL	SUMME
1	Afghanistan	2015	(null)	1	0.62
2	Afghanistan	(null)	(null)	2	10.35
3	Afghanistan	(null)	Travel	1	0.62
4	Afghanistan	(null)	Children	1	9.73

...

641	Zambia	(null)	(null)	4	27.38

641 selected

5.3

Code

```
SELECT manager_staff_id AS manager_id,
       store_id,
       SUM(amount) AS gesamterloes
FROM store
       JOIN staff USING (store_id)
       JOIN payment USING (staff_id)
GROUP BY CUBE (manager_staff_id, store_id)
ORDER BY manager_staff_id NULLS FIRST,
       store_id NULLS FIRST;
```

Ausgabe

	MANAGER_ID	STORE_ID	GESAMTERLOES
1	(null)	(null)	115640.89
2	(null)	1	44694.83
3	(null)	2	44551.46
4	(null)	3	6620.07
5	(null)	4	6657.14
6	(null)	5	13117.39
7	1	(null)	44694.83
8	1	1	44694.83
9	2	(null)	44551.46
10	2	2	44551.46
11	3	(null)	6620.07
12	3	3	6620.07
13	4	(null)	6657.14
14	4	4	6657.14
15	5	(null)	13117.39
16	5	5	13117.39

16 selected

5.4 -- selbe Abfrage, nur mit Grouping

Code

```
SELECT manager_staff_id AS manager_id,
       store_id,
       SUM(amount) AS gesamterloes,
       GROUPING(manager_staff_id) AS manager_gr,
       GROUPING(store_id) AS store_gr
FROM store
       JOIN staff USING (store_id)
       JOIN payment USING (staff_id)
GROUP BY CUBE (manager_staff_id, store_id)
ORDER BY manager_staff_id NULLS FIRST,
       store_id NULLS FIRST;
```

Ausgabe

	MANAGER_ID	STORE_ID	GESAMTERLOES	MANAGER_GR	STORE_GR
1	(null)	(null)	115640.89	1	1
2	(null)	1	44694.83	1	0
3	(null)	2	44551.46	1	0
4	(null)	3	6620.07	1	0
5	(null)	4	6657.14	1	0
6	(null)	5	13117.39	1	0
7	1	(null)	44694.83	0	1
8	1	1	44694.83	0	0
9	2	(null)	44551.46	0	1
10	2	2	44551.46	0	0
11	3	(null)	6620.07	0	1
12	3	3	6620.07	0	0
13	4	(null)	6657.14	0	1
14	4	4	6657.14	0	0
15	5	(null)	13117.39	0	1
16	5	5	13117.39	0	0

16 selected

Wenn die Funktion GROUPING() 1 ergibt, wurde dieser Wert mit anderen Werten zusammengefasst und ist daher nicht mehr direkt verfügbar, daher wird er als NULL angezeigt.
Wenn GROUPING 0 gibt, wurde der Wert als Kriterium für eine Gruppierung verwendet, falls trotzdem ein NULL Wert auftritt, stammt der Wert aus den zugrundeliegenden Daten.